

Before getting into the next pattern, let us see what is common between a proxy and an adapter.

Both proxy and adapters are intermediaries between providers and consumers. They both do some sort of optional pre-processing and post-processing. That's where the similarities end.

The difference between them can be summed up like this. The interface of the proxy and the provider is exactly the same, whereas the interfaces of the adapter and the third party providers are completely different.

Decorator

The last pattern that we look at in this part is the 'decorator', which helps in designing modular systems.

A decorator is also an intermediary between consumer and provider, just like the proxy and adapter. However, unlike them, the interface of the decorator is usually a superset of the interface of the provider. That way, a decorator decorates the provider. In other words, a decorator extends the interface of the provider and thereby offers additional functionality.

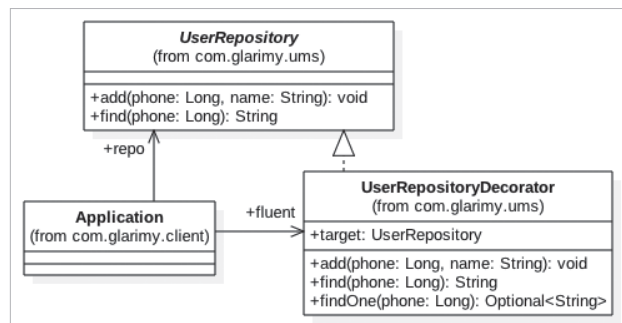


Figure 4: Decorator pattern

In the example depicted in Figure 4, the *UserRepositoryDecorator* is extending *UserRepository*:

```
public class UserRepositoryDecorator implements
UserRepository {
    ...
}
```

However, just like the proxy, it also holds a reference to the target provider through its interface. The original functionality will still be offered by the target provider while the decorator offers the additional functionality.

```
private UserRepository target;

public UserRepositoryDecorator(UserRepository target) throws
Exception {
    this.target = target;
}
```

The decorator can apply pre-processing and post-processing just like the proxy. At a minimum, the decorator has to implement all the interface methods and delegate the work to the target.

```
public void add(Long phone, String name) {
    target.add(phone, name);
}

public String find(Long phone) {
    return target.find(phone);
}
```

And finally, the *UserRepositoryDecorator* offers additional functionality *findOne()* that returns *Java 8 Optional*. This is useful for some consumers who are interested in using *UserRepository* along with lambdas and other functional programming constructs.

```
public Optional<String> findOne(Long phone) {
    String name = target.find(phone);
    if (name == null)
        return Optional.empty();
    return Optional.of(name);
}
```

An interesting point about decorators is that it is not mandatory for them to offer additional interface functions. In such a case, a decorator and a proxy are exactly the same.

Coming back to the *UserRepositoryDecorator*, it has to be used explicitly by the consumer in order to use the additional *findOne()* functionality.

```
Factory factory = new ObjectFactory("config.properties");
UserRepository repo = (UserRepository) factory.get("user.
repository");
repo.add(9731423166L, "Krishna");
UserRepositoryDecorator fluent = new
UserRepositoryDecorator(repo);
fluent.findOne(9731423166L).ifPresent(name -> System.out.
println("Found " + name));
```

You may not like the *UserRepositoryDecorator* in the current shape, since it is a concrete class with additional functionality. Instead, you may want to have an interface with the additional functionality and thereby a concrete class.

For example, you may want the *UserRepositoryDecorator* as an interface that extends